Processing and Visualising Large DEMs on a Laptop

GeoGuessers | Spatial Informatics, final presentation

Swayam Goyal (2024111012) Siddharth Choudhary (2024111021)
Yakkala Viswanadha Kartikeya (2024111033)

The problem

- High-resolution DEMs are huge: our 100 km × 100 km Himalayan area at 1 m is **10.8 billion cells, 43 GB** as float32.
- One laptop: Core Ultra 9 185H (6 performance + 8 efficiency + 2 low-power cores), **16 GB RAM**, RTX 4060 (8 GB), NVMe SSD.
- Users want slope, aspect, hillshade, roughness . . . for whatever they are looking at, *now*.

Professor's direction: process and display only what is in view, at the resolution the screen needs, from a pyramid; choose CPU or GPU by the measured bottleneck; integrate with GDAL/QGIS.

43 GB
grid at 1 m

16 GB
RAM

60 Hz
what a map viewer must feel like

Research gaps (from our literature review) and questions

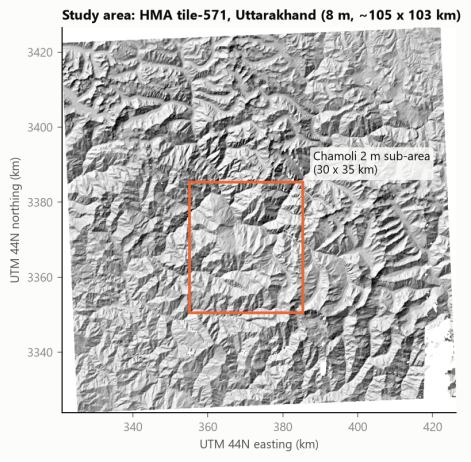
G1 No study reports **where standard GDAL/QGIS terrain tools stop being practical** on one laptop, split by stage (read, compute, CPU↔GPU transfer, write, render).

G2 No study computes **derivatives on demand from pyramid levels**, or measures the slope error of a coarse level against the time it saves.

G3 Viewshed and terrain distance accuracy (stretch; not done).

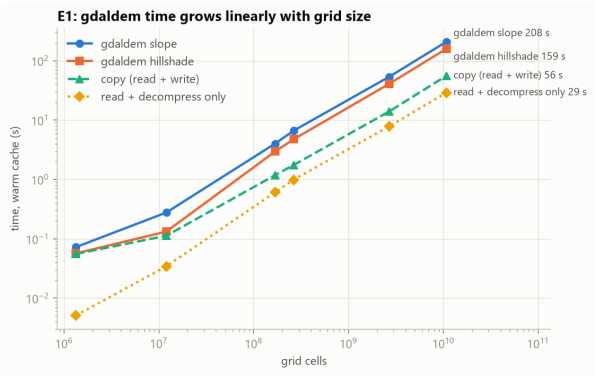
1. Where is the real bottleneck, for processing and for display? **E1–E4**
2. What does computing on demand from the pyramid cost in accuracy, and save in time? **E5**
3. Can on-demand computation keep up with continuous pan and zoom? **E6**

Data and method



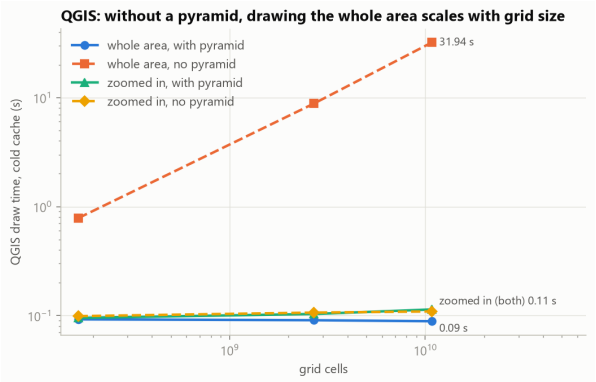
- **Real data:** Chamoli 2 m (264 M cells), HMA 8 m, Copernicus 30/90 m; UTM 44N.
- **Synthetic 2 m and 1 m** versions of the whole area, for timing only (never accuracy).
- Cloud-Optimized GeoTIFFs: tiled, ZSTD, overviews = the **pyramid**.
- **Protocol:** 1 warm-up + 5 measured runs, median and IQR; cold (file cache flushed) and warm; peak memory of the measured process; one row per run in `results/runs.csv`.
- Every number in this talk is reproducible from the repository's scripts.

E1: standard tools do not fail, they become slow



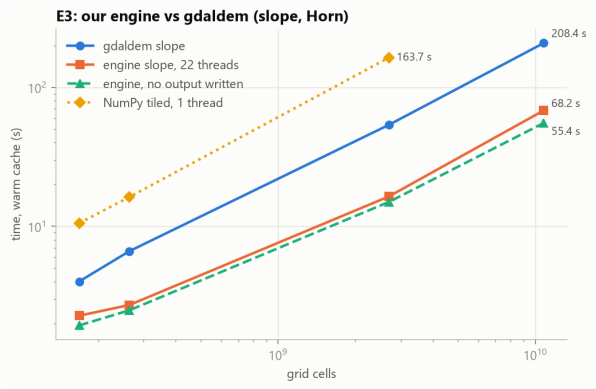
- `gdaldem` streams the grid: peak **0.87 GB** even at 1 m.
- Time grows linearly, **19–25 ns per cell**: one slope pass over 1 m takes **3.5 min**.
- Bottleneck: one CPU thread computing (73% of the time; 1 of 22 threads), not disk (14%).
- File layout matters by up to ∼40%; computation still dominates.

E1: for display, the pyramid is everything



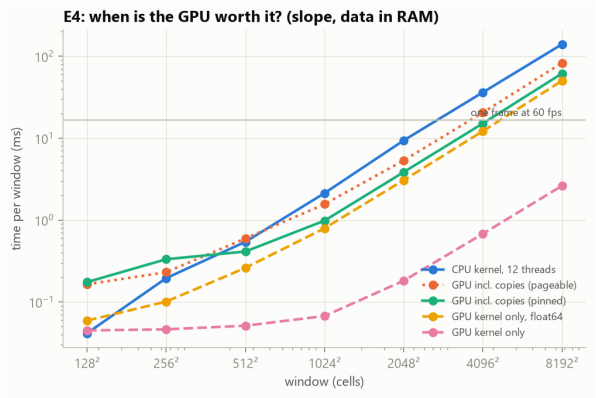
- QGIS redrawing the whole 1 m area: **32 s** without a pyramid.
- With a pyramid: **0.09 s** at every resolution.
- Display is bound by reading full-resolution data; the pyramid removes it.

E3: our tiled, parallel CPU engine



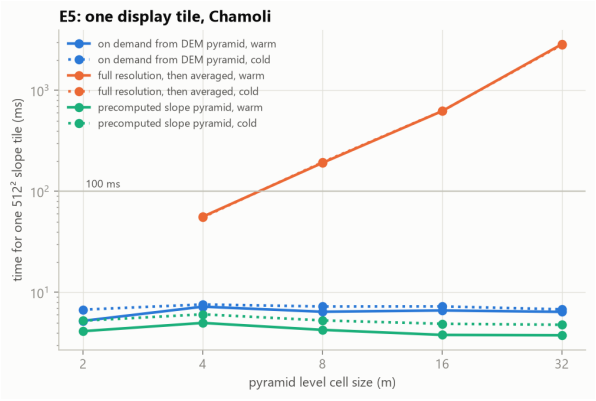
- Windows of 2048^2 with a 1-cell halo; reader threads → Numba kernel → writer.
- Slope **2.1–3.5× faster than gdaldem** (1 m: 60 s vs 208 s); all 10 gdaldem configurations.
- Memory flat at **1.4 GB**; whole-grid NumPy runs out of memory from 2 m synthetic.
- Three products in one pass: 3.4–4.2× fewer seconds than three gdaldem runs.
- 12 performance-core threads beat all 22.

E4: the GPU computes 50× faster, the run is 1.0–1.4× faster



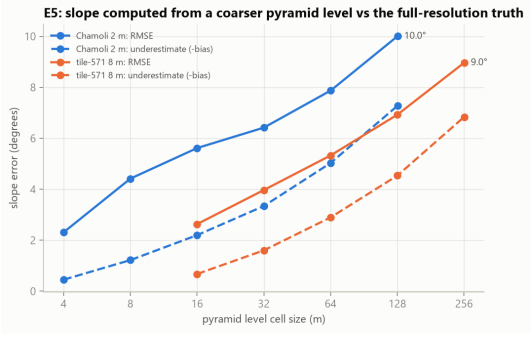
- GPU kernel: 1.1 s for 10.8 B cells (CPU kernel 34 s). Whole 1 m run: **43.8 s** vs 60.3 s.
- Copies are 95% of the GPU's own time; the GPU wins only from 512^2 cells per window.
- Each fix moves the bottleneck: **compute** → **decompress** → **write**.
- GPU never stressed: ≤63 °C, ≤2 GiB of 8, no setting changed.

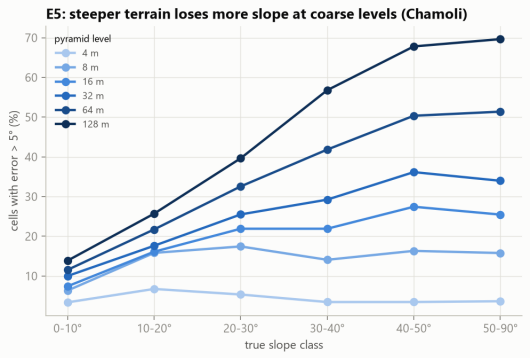
E5 (G2): on demand, every tile costs the same at every zoom



- Slope for a 512×512 tile from pyramid level k: **5–8 ms** at every level.
- Exact (full resolution, then averaged): 56 ms at 4 m … **2.9 s** at 32 m.
- On demand saves **8–450**×, with no precomputed storage (a slope pyramid costs 1.0 GB per derivative).

E5 (G2): the price is slope that is too low



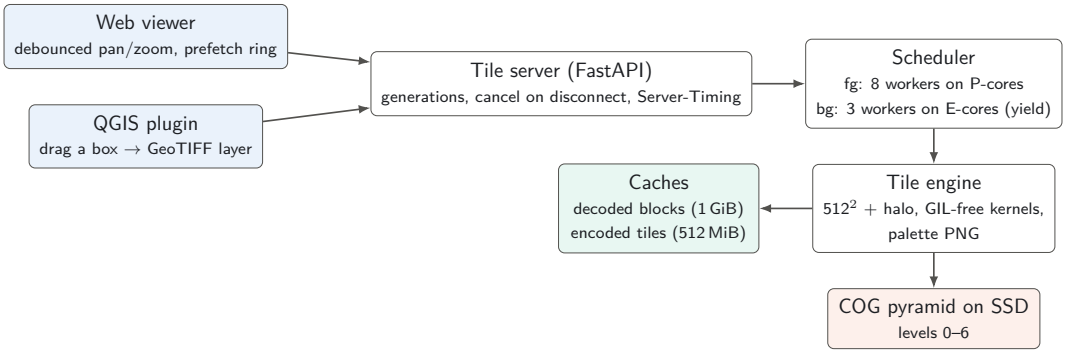


RMSE (Chamoli 2 m): **2.3**° one level up → 6.4° four → 10.0° six; almost all underestimate, worst on steep ground. Rule of thumb: ≈2.5° one level up, +1.5° per further level. Aspect survives (median 2–8°). A pyramid level of good data beats an independent DEM of the same resolution.

From findings to a viewer design

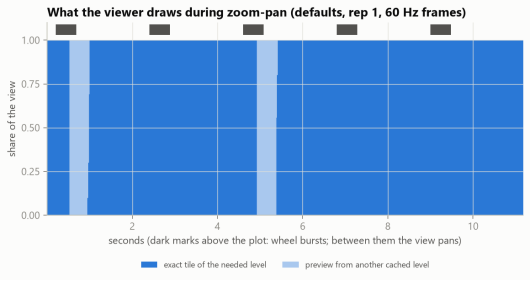
Finding	Design decision
Display needs a pyramid (E1)	Level whose cells fit one screen pixel
A tile costs the same at every level (E5)	Fixed 512^2 tile lattice per level, aligned to the COG blocks
Decoding, not arithmetic, dominates (E3, E5)	Shared cache of decoded blocks; cache of encoded tiles
Efficiency cores slow the foreground (E3)	Visible tiles on P-cores, prefetch on E-cores
GPU only wins on big windows (E4)	CPU for tiles; GPU left for bulk jobs
Coarse levels underestimate slope (E5)	Expected error shown; “exact” mode on request

The system


Web viewer
debounced pan/zoom, prefetch ring
QGIS plugin
drag a box → GeoTIFF layer
Tile server (FastAPI)
generations, cancel on disconnect, Server-Timing
Scheduler
fg: 8 workers on P-cores
bg: 3 workers on E-cores (yield)
Caches
decoded blocks (1 GiB)
encoded tiles (512 MiB)
Tile engine
512^2 + halo, GIL-free kernels, palette PNG
COG pyramid on SSD
levels 0–6

How the viewer behaves

- **Zoom:** every wheel tick moves the camera at once; cached coarser tiles stand in. One commit 150 ms after the last tick: new generation, stale requests aborted (the server cancels them).
- **Pan:** newly visible tiles requested immediately.
- **Prefetch:** after a commit, the 8 neighbouring views at low priority, direction of travel first, at most 2 in flight; stops at the first input of a gesture.
- **Shift + drag** zooms to a box.

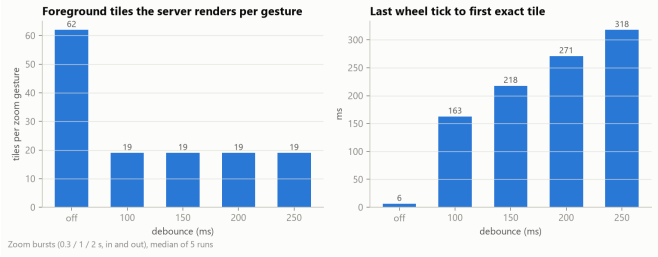


Zoom-pan trace: dark = exact tile of the needed level, light = preview from another cached level.

E6: how we measured the viewer

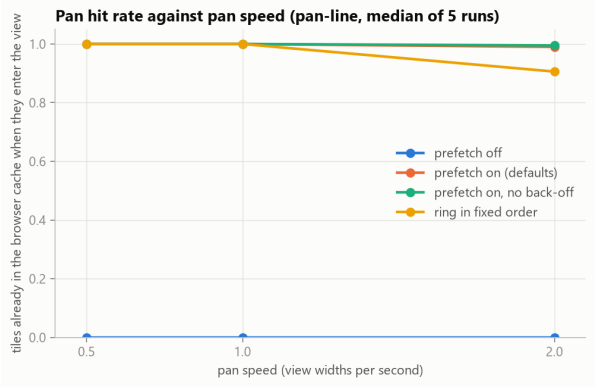
- **Virtual client:** the browser's policy ported to Python, reading the same `policy.json`; real HTTP, 6 connections like a browser, 60 Hz frames, aborts close the connection.
- **17 seeded traces:** pans at 0.5/1/2 view widths per second, reversals, random walk, wheel bursts, notched wheel, zoom-pan, nested boxes.
- **24 configurations, one factor at a time; 1,068 runs**, each on a fresh server (empty caches).
- **Checked against real Chrome** (DevTools protocol): same requests and commits; view-complete within 10%.

E6: the debounce cuts server work 3.3×



- Every tick commits: **62** server tiles per zoom; debounced: **19**.
- First exact tile **63–71 ms** after the debounce (gate: $<$100 ms).
- **Notched wheel:** 100 ms = as bad as none (36 tiles); 150 ms coalesces (15).
- $\Rightarrow$ default 150 ms.

E6: prefetch makes panning seamless



- After each commit the 8 neighbouring views load at low priority, direction of travel first.
- **98–100%** of tiles entering the view are already cached, up to 2 view widths per second.
- Without prefetch: 0% on a straight pan, ∼50% when the pan turns back.
- Cost: 2–5 tiles fetched per tile used, on the efficiency cores (∼3 CPU-s per 5 s pan).

E6 gate 1: prefetch does not slow the visible tiles

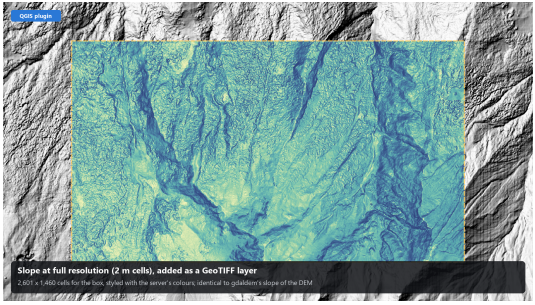


On identical tiles the server p95 is **90 ms with prefetch vs 95 ms without**: prefetch stops at the first input of a gesture, so only 3% of foreground tiles ever overlap it. Every one of 122,215 prefetch jobs ran on an efficiency core.

E6: what sets the remaining latency

- **The browser's 6 connections:** client p95 $\approx$ server p95 + 40 ms; 4, 6 or 8 foreground workers give the same client latency.
- **Block decoding:** without the block cache, 6$\times$ more blocks decoded and p95 +40%.
- **Not the derivative:** slope, aspect, hillshade within $\sim$10%; precomputed slope fastest on the server.
- **Not the disk:** a flushed file cache changes nothing.
- A first back-off rule (pause prefetch if foreground p95 $>$ 100 ms) sat exactly at the cold-start latency and switched prefetch off at random; 250 ms fixes it.

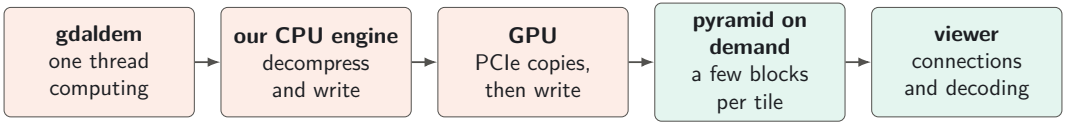
Demo



QGIS plugin: a 2 m slope box from the same server.

- Live: `python -m server`, then `http://127.0.0.1:8765/viewer/`
- Wheel zoom, pan, product switch, sun direction, Shift + drag box, status panel (level, cell size, expected error)
- Fallback video (58 s): `report/demo/geoguessers_demo.mp4`

The bottleneck story (G1)



- On a laptop, standard tools stop being practical by being **slow, not by running out of memory**.
- Once computation is parallel, moving and decoding data is the cost; the GPU only shifts it.
- Reading **only what is in view, at the needed level** sidesteps the chain: a tile costs milliseconds at any zoom.

Limitations and future work

Limitations

- One laptop, one study region (steep terrain: larger slope errors than flat land).
- Synthetic 2 m/1 m grids used for timing only.
- E6: one client, short scripted traces, cold server start.
- Coarse-level bias not corrected.

Future work

- G3: viewshed (R3 vs XDraw vs `gdal_viewshed`) and terrain distance (Dijkstra vs A*).
- Per-level slope bias correction.
- Several clients and long sessions; remote (cloud) COGs.
- GPU tile path for bulk exports; Cesium globe.

Takeaways

1. **G1:** on one laptop the bottleneck moves from single-threaded computation to decompressing and writing data; a GPU computes 50× faster but the run is only up to 1.4× faster.
2. **G2:** derivatives on demand from the pyramid cost 5–8 ms per tile at any zoom, 8–450× less than exact, for $\approx 2.5^\circ + 1.5^\circ$/level slope RMSE (an underestimate).
3. **Viewer:** with a 150 ms debounce and an efficiency-core prefetch ring, on-demand computation keeps up with continuous pan and zoom; the browser's connections and block decoding set the latency.

Code, data manifests, results and reports: `https://github.com/Swayamgoyal/Geoguessers`

Thank you

Questions?